UNIVERSIDAD TECNOLÓGICA DE CHIHUAHUA



Entornos Virtuales y Negocios Digitales

Programación de Aplicaciones Web Progresivas

Reporte: Aplicaciones Web Progresivas (PWA)

PRESENTA:

Ángel Iván López Delgado

Chihuahua, Chih., a 4 de septiembre de 2026

1. Introducción

Durante años el desarrollo de software para usuarios finales estuvo partido en dos caminos que casi no se tocaban: la aplicación nativa, instalable y con acceso al hardware del dispositivo, y el sitio web, accesible desde cualquier navegador pero limitado a la ventana del mismo y dependiente de la red. Elegir uno implicaba renunciar a las ventajas del otro, y para muchas organizaciones significaba mantener tres bases de código distintas: una para Android, una para iOS y una para la web.

Las aplicaciones web progresivas, conocidas como PWA por sus siglas en inglés (*Progressive Web Apps*), nacen como respuesta a esa división. Se trata de aplicaciones construidas con tecnologías web estándar -HTML, CSS y JavaScript- que incorporan un conjunto de capacidades que históricamente eran exclusivas de las aplicaciones nativas: funcionamiento sin conexión, instalación en la pantalla de inicio, notificaciones push y acceso a determinadas APIs del dispositivo.

El presente reporte documenta qué es una aplicación web progresiva, cómo surgió el concepto, en qué escenarios se utiliza, y cuáles son sus ventajas y desventajas frente a las alternativas nativa e híbrida.

2. ¿Qué es una aplicación web progresiva?

Una PWA es una aplicación web que aprovecha las capacidades modernas de los navegadores para ofrecer una experiencia comparable a la de una aplicación instalada, sin dejar de ser un sitio web accesible mediante una URL.

El adjetivo *progresiva* proviene del principio de **mejora progresiva** (*progressive enhancement*): la aplicación funciona en cualquier navegador, y a medida que el navegador y el dispositivo soportan más capacidades, la experiencia se enriquece. Un usuario con un navegador antiguo obtiene un sitio web funcional; uno con un navegador moderno obtiene, sobre esa misma base, instalación, modo sin conexión y notificaciones.

2.1 Componentes técnicos

Tres piezas sostienen el modelo:

Service Worker. Es un archivo de JavaScript que el navegador ejecuta en segundo plano, en un hilo independiente del de la página. Actúa como un proxy entre la aplicación

y la red: puede interceptar las peticiones, responderlas desde una caché local y decidir qué se descarga y qué no. Es el componente que hace posible el funcionamiento sin conexión, la sincronización en segundo plano y la recepción de notificaciones push.

Manifiesto de la aplicación (*Web App Manifest*). Es un archivo en formato JSON que describe la aplicación ante el sistema operativo: nombre, iconos, color de tema, orientación y modo de presentación. Es lo que permite que la aplicación se instale en la pantalla de inicio y se abra en su propia ventana, sin la barra de direcciones del navegador.

HTTPS. Los service workers solo se registran en orígenes seguros. La razón es que un componente capaz de interceptar todo el tráfico de una aplicación sería un riesgo grave si pudiera inyectarse a través de una conexión sin cifrar.

2.2 Características esperadas

De manera general se considera que una aplicación web es progresiva cuando cumple con:

- **Capacidad de respuesta:** se adapta a distintos tamaños de pantalla.
- **Independencia de la conectividad:** funciona total o parcialmente sin conexión.
- **Instalabilidad:** puede añadirse a la pantalla de inicio del dispositivo.
- **Seguridad:** se sirve exclusivamente por HTTPS.
- **Enlazabilidad:** cada vista tiene una URL que puede compartirse.
- **Descubribilidad:** los motores de búsqueda pueden indexarla.
- **Actualización automática:** el service worker mantiene el contenido al día sin intervención del usuario.

3. Historia

3.1 Antecedente: el iPhone y las aplicaciones web (2007)

Al presentar el iPhone en 2007, Steve Jobs anunció que el modelo de desarrollo para el dispositivo serían las aplicaciones web ejecutadas en Safari, no las aplicaciones instaladas. La propuesta no prosperó: las capacidades de los navegadores de la época eran insuficientes y la App Store, lanzada en 2008, terminó imponiendo el modelo de aplicación nativa que dominó la década siguiente. Aun así, la idea de fondo -que la web fuera la plataforma de aplicaciones- quedó planteada.

3.2 El acuñamiento del término (2015)

El término *Progressive Web App* fue acuñado en 2015 por el ingeniero de Google **Alex Russell** y la diseñadora **Frances Berriman**. El texto que lo introdujo, *Progressive Web Apps: Escaping Tabs Without Losing Our Soul*, describía un tipo de aplicación que empezaba como un sitio web común y, conforme el usuario la usaba de manera recurrente, se iba "ganando" un lugar como aplicación instalada.

Lo relevante es que el término llegó después de la tecnología: la especificación de los service workers ya se estaba desarrollando y Chrome comenzó a implementarlos en 2014-2015. Las PWA no fueron un producto lanzado, sino un nombre puesto a un conjunto de capacidades que estaban convergiendo.

3.3 Adopción y casos tempranos (2016-2018)

Entre 2016 y 2018 varias empresas de gran escala publicaron versiones PWA de sus productos, en muchos casos dirigidas a mercados con conexiones lentas y dispositivos de gama baja:

- **Twitter Lite** (2017), que redujo drásticamente el peso de la aplicación frente a la nativa.
- **Starbucks**, cuya PWA permitía armar un pedido sin conexión.
- **Pinterest**, que rehízo su sitio móvil como PWA y reportó aumentos significativos en interacción e ingresos publicitarios.
- **Trivago**, **Forbes**, **Uber** y **AliExpress**, entre otros.

Google impulsó el modelo con herramientas de auditoría como **Lighthouse**, que evalúa si un sitio cumple los criterios de una PWA.

3.4 El soporte en Apple y la situación actual

El punto de inflexión para la adopción general fue **iOS 11.3, publicado en marzo de 2018**, cuando Safari incorporó soporte para service workers y para el manifiesto de aplicación web. Hasta ese momento, la mitad del mercado móvil en varios países quedaba fuera del alcance de una PWA.

A partir de entonces el modelo se consolidó como una opción de uso corriente, aunque el soporte sigue siendo desigual: Apple ha mantenido limitaciones en cuotas de almacenamiento, notificaciones push (habilitadas en Safari para iOS hasta 2023) y acceso a APIs de hardware. Buena parte de las críticas al modelo apuntan a esa

disparidad más que a la tecnología en sí.

4. Usos

Las PWA resultan especialmente adecuadas en los siguientes escenarios:

Comercio electrónico. La fricción de instalar una aplicación hace que muchos usuarios abandonen antes de comprar. Una PWA permite entrar desde un enlace, comprar de inmediato e instalar después si el usuario vuelve.

Medios y contenido editorial. Los sitios de noticias necesitan ser indexables por buscadores y compartibles por enlace, algo que una aplicación nativa no ofrece, pero también carga rápida y lectura sin conexión.

Herramientas internas y aplicaciones de línea de negocio. Distribuir software interno por las tiendas de aplicaciones implica procesos de revisión y gestión de dispositivos. Una PWA se despliega en un servidor y se actualiza para todos al instante.

Mercados emergentes y conectividad limitada. Es el caso de uso que originó buena parte de la adopción: dispositivos con poco almacenamiento y redes intermitentes, donde descargar decenas de megabytes no es viable.

Aplicaciones de uso ocasional. Servicios que el usuario utiliza pocas veces al año difícilmente justifican una instalación permanente.

Productividad y trabajo colaborativo. Servicios como correo, mensajería o edición de documentos aprovechan la instalación en escritorio para tener ventana propia y notificaciones, sin dejar de ser accesibles desde el navegador.

5. Ventajas

#	Ventaja	Descripción
1	Una sola base de código	El mismo desarrollo sirve para Android, iOS, escritorio y web, con la consiguiente reducción de costo y tiempo.
2	Sin fricción de instalación	El usuario accede desde una URL; la instalación es opcional y posterior.
3	Actualización inmediata	Se publica en el servidor y todos los usuarios reciben la versión nueva, sin esperar revisiones de tienda ni depender de que actualicen.

#	Ventaja	Descripción
4	Independencia de las tiendas	No hay proceso de aprobación ni comisión sobre las transacciones.
5	Menor peso	Suelen ocupar una fracción del espacio de una aplicación nativa equivalente.
6	Descubribilidad	Al ser sitios web, los motores de búsqueda las indexan y su contenido es enlazable.
7	Funcionamiento sin conexión	El service worker permite seguir operando con red intermitente o nula.
8	Seguridad por diseño	El requisito de HTTPS es obligatorio, no opcional.

6. Desventajas

#	Desventaja	Descripción
1	Acceso limitado al hardware	Sensores, Bluetooth, NFC y funciones avanzadas de cámara tienen soporte parcial o inexistente según el navegador.
2	Soporte desigual entre plataformas	El comportamiento en iOS ha ido históricamente por detrás del de Android, con límites en almacenamiento y notificaciones.
3	Menor visibilidad comercial	No aparecen en App Store ni Google Play, lo que elimina un canal de descubrimiento y, para algunos usuarios, una señal de confianza.
4	Rendimiento inferior en casos exigentes	En aplicaciones con gráficos intensivos o procesamiento pesado, la aplicación nativa sigue siendo superior.
5	Consumo de batería	La ejecución sobre el motor del navegador es menos eficiente que el código nativo.
6	Complejidad de la caché	Una estrategia de caché mal diseñada puede dejar a los usuarios con versiones desactualizadas y es una fuente frecuente de errores difíciles de reproducir.
7	Persistencia no garantizada	El sistema operativo puede liberar el almacenamiento de la aplicación si necesita espacio.
8	Desconocimiento del usuario	Muchos usuarios no identifican la opción de instalar desde el navegador, lo que reduce la tasa de instalación.

7. Conclusiones

Las aplicaciones web progresivas no sustituyen a las nativas: acortan la distancia entre ambas. Para la mayoría de los productos -comercio, contenido, servicios, herramientas internas- la diferencia de capacidades ya no justifica el costo de mantener desarrollos separados para cada plataforma, y el modelo progresivo permite además que el usuario decida si quiere instalar o no.

La decisión debe tomarse según el caso: cuando el producto depende de acceso profundo al hardware o de rendimiento gráfico exigente, la aplicación nativa sigue siendo la opción correcta. Cuando lo que se busca es alcance, actualización rápida y costo de desarrollo contenido, la PWA es hoy la alternativa más razonable.

Conviene, por último, señalar que el principal obstáculo del modelo no ha sido técnico sino comercial: las limitaciones impuestas por los ecosistemas cerrados han condicionado su adopción tanto o más que las capacidades de la propia tecnología.

8. Referencias

- Russell, A. y Berriman, F. (2015). *Progressive Web Apps: Escaping Tabs Without Losing Our Soul*. infrequently.org
- MDN Web Docs. *Progressive web apps*. developer.mozilla.org/en-US/docs/Web/Progressive_web_apps
- Google Developers. *Learn PWA*. web.dev/learn/pwa
- W3C. *Web Application Manifest* y *Service Workers* (especificaciones).
- WebKit Blog (2018). *Release Notes for Safari Technology Preview / iOS 11.3*.